



CODE SPORT

In this month's column, we continue our discussion on information retrieval.

In last month's column, we explored distributed algorithms to construct an inverted index, which is the basic data structure used in all information retrieval systems. We looked at how Map-Reduce techniques can be used in the construction of an inverted index and how incremental construction of the inverted index can be done. Document collections typically keep growing, as seen in many of the common cases of information retrieval systems such as the World Wide Web or document repositories of technical reports/medical records, etc. The major impact of an ever-growing document collection is the increase in the size of the dictionary and the postings lists, which constitute the inverted index data structure.

We often encounter situations in which the footprint of the inverted index cannot fit into main memory. This results in slow look-ups on the inverted index which, in turn, slows down the user queries. One way of mitigating this problem is to compress the inverted index and keep the compressed form in the main memory. In this column, we focus our attention on how compression schemes can be applied to the inverted index effectively.

Before going into compression schemes, let us look at a couple of empirical laws that generally hold true for IR systems. One is known as Heaps' Law and another is known as Zipf's Law. Heaps' Law gives an approximate estimate for the number of distinct terms in a document or set of documents as a function of the size of the document or document collection, as the case may be. Let us define ' V ' to be the vocabulary of the document collection. ' V ' is nothing but the number of distinct terms in the document collection. Let ' N ' be the total number of tokens in the collection (note that N counts all tokens, not unique tokens). Then Heaps' Law can be stated as:

$$V = K N^\beta$$

where K and β are constants. Typically, β is around

0.4-0.6 and K is between 10-100. Given β is around 0.5, we can estimate the number of distinct terms to be approximately the square root of N , where N is the total number of tokens in the collection. Recall that the dictionary in the inverted index contains the vocabulary of the collection. The implication behind Heaps' Law is that as the number of documents in the collection increases, the dictionary size also continues to increase rather than saturating at a maximum vocabulary size, and the sizes of the dictionary are typically larger for large document collections. This makes it difficult to maintain the entire dictionary in memory for large collections and hence the need to compress it.

Before understanding how to compress the inverted index, let us look at the second empirical law associated with information retrieval—Zipf's Law. This deals with the frequency of a term in the collection. Note that the frequency is the number of times a term occurs in the collection, and a frequency table lists the terms and their frequencies in descending order. Zipf's Law states that for any given collection, the frequency of a term in the collection is inversely proportional to its rank in the frequency table. This means that the second most frequent word will appear only half the number of times as the most frequent word in the collection, and the third-most frequent word in the collection will appear only one-third the number of times as the most frequent word appears. The implication behind Zipf's Law is that a term's frequency declines rapidly with its rank in the frequency table. This, in turn, implies that a few distinct terms typically account for a large number of tokens in a document collection. What does this mean in the context of the increasing size of document collections?

The point to note here is that as the frequency of a term falls with its rank in the frequency table, it allows us to omit certain terms. For instance, we can choose to omit terms that are very rare, under the